Bench wiring sheet - protocol emulator on the MachXO3L kit (3 Nov 2026)

One wire per row: **FROM** one end **TO** the other, **AND** a third end where the wire has one (a pull-up, a second board). Kit pins are written **J4-n (ball)**: n is the numbered hole on header J4, the ball name (K12...) is the FPGA's own label printed beside it, the same connection under a second name, for checking by continuity. **Go by the hole number**: the ball letters H and M (H16 = hole 25, M16 = hole 10) are easy to misread. **Engine pin** numbers (0-19) are what the chip's programs call that pin. Everything is **3.3 V logic** (FPGA and ESP32 pins are not 5 V tolerant); one common ground between every board.

1. Core: kit to ESP32 host. Fit these eight first, then rerun.ps1 (command r): ID reads 0x0022, 0 problems.

FROM	TO	AND (third end)	What it is
J4-3 (K12)	ESP32 GPIO 5		CS, host SPI
J4-5 (M14)	ESP32 GPIO 23		MOSI, host SPI
J4-7 (L14)	ESP32 GPIO 19		MISO, host SPI
J4-9 (M15)	ESP32 GPIO 18		SCK, host SPI
J4-11 (GND)	ESP32 GND	every other board's GND	common ground
J4-4 (K13)	ESP32 GPIO 4		IRQ: high while any engine has data
J4-6 (N14)	ESP32 GPIO 16		engine pin 0: engine A's UART out, into ESP32 Serial2 RX
J4-26 (G15)	ESP32 GPIO 17		engine pin 15: the ESP32 sends test pulses into the chip (glitch filter)

J4-1 and J4-2 are the kit's 3.3 V (printed IO1, the bank supply, not a ball name): use for pull-ups only, never join them to the ESP32's 3.3 V. Kit: **bash fpga/machxo3/build.sh load** (volatile SRAM, never the NVCM). ESP32 (the scripts find its port themselves): **esp32\upload_and_watch.ps1 -Sketch chip_test**.

2. I2C bus (engine pins 2 = SCL, 3 = SDA). ESP32 as the slave: rerun.ps1 -Cmd i. UNO R3 as the slave: -Cmd n.

FROM	TO	AND (third end)	What it is
J4-10 (M16) SCL	ESP32 GPIO 22 or UNO A5	5.1 k resistor to 3.3 V	SCL, engine pin 2
J4-13 (L15) SDA	ESP32 GPIO 21 or UNO A4	5.1 k resistor to 3.3 V	SDA, engine pin 3
UNO GND	common ground		only when the UNO is the slave; its own 5 V pull-ups are switched off by its sketch
3.3 V for the two pull-ups	J4-1 or the UNO's 3.3 V pin		one source, not both

Never both slaves at once (both answer at address 0x50): unplug the UNO's A4 and A5 for -Cmd i. UNO sketch: arduino/i2c_slave_nano on COM11.

3. SPI master to the 74HC595 (engine pins 4 to 7). rerun.ps1 -Cmd s.

FROM	TO	AND (third end)	What it is
J4-14 (L16)	595 pin 11 SRCLK		SCK, engine pin 4
J4-15 (K14)	595 pin 14 SER		MOSI, engine pin 5
J4-16 (K16)	595 pin 9 QH'		MISO, engine pin 6: the 595's serial out comes back
J4-17 (K15)	595 pin 12 RCLK		CS / latch, engine pin 7
595 pin 16 VCC	3.3 V		power
595 pin 8 GND	common ground		power
595 pin 13 OE	GND		outputs enabled
595 pin 10 SRCLR	3.3 V		not clearing
595 pins 15, 1, 2, 3, 4, 5, 6, 7 (QA..QH)	LED long leg	LED short leg -> 330 ohm -> GND	red, yellow or green only (white/blue stay dim at 3.3 V; the flat-topped ones never lit); the run ends on 0x55 so QA (pin 15) is high

Before the rebuild the LEDs stayed dark with the chip side proven good: meter pins 16, 8, 13, 10 first, then pin 15 after a run, then the QH' lead.

4. Every J4 hole the chip uses (for anything new)

J4 holes	Balls	Engine pins	Direction
6, 8, 10, 13, 14, 15, 16, 17	N14, N16, M16, L15, L16, K14, K16, K15	0, 1, 2, 3, 4, 5, 6, 7	in and out (open drain able)
18, 19, 20	J14, H14, J15	8, 9, 10	out only
23, 24, 25	J16, H15, H16	11, 12, 13	out only
26, 27, 28, 29, 30	G15, G16, F15, F16, E15	15, 16, 17, 18, 19	in only
1, 2	3.3 V		
11, 12	GND		

Holes and balls pair up in order: hole 6 is ball N14 (engine pin 0), hole 8 is N16 (pin 1), and so on along each row. Kit LEDs: P16 lights for IRQ, N15 is the heartbeat (they light when driven low).
Cyclone IV board (black EP4CE6E22C8N core board): everything on the right-hand header, pins 38-72, both bank jumpers at 3.3. CS 38, SCK 39, MOSI 42, MISO 43, IRQ 44 to the same ESP32 GPIOs as section 1; engine pin 0 = 46 (to GPIO 16), pins 1-7 = 49 50 51 52 53 54 55 (I2C SCL 50 / SDA 51; SPI 52-55), pins 8-13 = 58 59 60 64 65 66, pin 15 = 67 (from GPIO 17), 16-19 = 68-71. Clock 50 MHz on pin 24, LEDs 1 (IRQ) and 2 (heartbeat). Needs a USB Blaster on the JTAG header (ordered 3 Nov, due 5-6 Nov): bash fpga/cyclone4/build.sh load. Sketch: CHIP_HZ = 50e6. This board runs three engines (ID 0x0023); the kit two.
Lesson of 4 Nov night: an instrument on the bus is powered or unclipped, never off and attached. The logic analyser switched off with its probes still on SCL and SDA clamped both lines through its input diodes: the EEPROM NAKed its address, writes failed, reads came back FD FD FD FD, while the chip's own sniffer showed the bus faithfully. Powered on, everything passed again.

What is plugged in right now (4 Nov 2026, midday): no double booking

A new program may only use a FREE hole; moving a lead means updating this list. TAKEN: hole 1 (IO1 3.3 V: the keypad's four 10 k pull-ups); holes 3, 4, 5, 7, 9 (the host, ESP32 or Pico: CS, IRQ, MOSI, MISO, SCK); hole 6 (engine pin 0, the serial line: ESP32 GPIO 16 reads the chip's UART out, or with the Pico as host its GP0 sends to the chip for pico/uart_rx.py, the ESP32 lead out of the hole then, 4 Nov night); holes 10 and 13 (I2C SCL/SDA: the AT24C256 module, section 9, and from 5 Nov the DS3231 and MCP4725 modules on the same wires, sections 12-13; before that the UNO's A5 and A4); holes 14-17 (keypad columns, tail 5-8; the 74HC595 is out of circuit since 4 Nov; the W25Q32 flash takes these four holes, section 11, columns out first); hole 26 (ESP32 GPIO 17 test pulses, the regression needs it); holes 27-30 (keypad rows, tail 1-4, 10 k to hole 1). hole 19 (engine pin 9, H14: the IR emitter's signal lead since 4 Nov evening); hole 8 (engine pin 1: the KY-022 IR receiver's S lead since 4 Nov afternoon; the HC-SR04 echo needs it later, one at a

time); hole 2 (second 3.3 V: the receiver's +). **FREE**: holes 18 and 20 (engine pins 8 and 10, out only; 18 pencilled for the HC-SR04 trigger and the DALI TX, section 10, planned); holes 23, 24, 25 (engine pins 11-13, out only). ESP32 pins taken: 4, 5, 16, 17, 18, 19, 23, plus 21 and 22 only if it is the I2C slave; VIN (5 V) is free for the HC-SR04. All two-way engine pins 0-7 are spoken for while the keypad and the IR receiver are on; spi_master.asm needs four consecutive pins with the third able to read: pins 4-7 only on this kit.
If it doesn't answer: esp32 rerun.ps1 -Cmd p (lead probe: a FLOATING pin = lead open or wrong hole; ID 0x0000 with MISO driven = CS/SCK/MOSI not arriving; 0xFFFF = MISO open), -Cmd q (IRQ held high 15 s), -Cmd b (I2C wire check: names the chip pin each I2C lead reaches), -Cmd k (GPIO 22 held low 30 s to meter), -Cmd l (repeating ID reads for the oscilloscope), -Cmd y (keypad: names every key pressed; every key 'pressed' = row pull-ups not reaching J4-27..30), -Cmd e (live level of engine pins 15-19 for 30 s, a line on every change: a row with its 10 k in place reads 1). Writes working but reads all zero = MISO lead's J4 end in a hole held low (11/12 GND, 18-25 driven low). The kit forgets its build on any power blip: reload with bash fpga/machxo3/build.sh load.

Order of work after replugging

1. Section 1 only. Load the kit, upload the sketch, rerun.ps1 (command r): ID 0x0022, 0 problems. 2. Add section 2: rerun.ps1 -Cmd i (ESP32 slave) or -Cmd n (UNO slave). 3. Add section 3: rerun.ps1 -Cmd s. 4. Keypad: section 6 (595 leads out), rerun.ps1 -Cmd y.

6. Keypad (rerun.ps1 -Cmd y), IR receiver and ultrasonic: the Elegoo kit parts (4 Nov 2026)

Keypad tail, keys facing you and the tail at the bottom, pin 1 on the left: pins 1-4 are the **rows** top to bottom, 5-8 the **columns** left to right. The chip pulls one column low at a time and reads the rows, so each row needs a 10 k pull-up. The 595 shares J4-14 to 17: unplug its four leads first.

FROM	TO	AND (third end)	What it is
keypad pin 1 (row 1: 1 2 3 A)	J4-27 (G16)	10 k resistor to J4-1 (3.3 V)	row 1, engine pin 16
keypad pin 2 (row 2: 4 5 6 B)	J4-28 (F15)	10 k resistor to J4-1	row 2, engine pin 17
keypad pin 3 (row 3: 7 8 9 C)	J4-29 (F16)	10 k resistor to J4-1	row 3, engine pin 18
keypad pin 4 (row 4: * 0 # D)	J4-30 (E15)	10 k resistor to J4-1	row 4, engine pin 19
keypad pin 5 (column 1: 1 4 7 *)	J4-14 (L16)		column 1, engine pin 4
keypad pin 6 (column 2: 2 5 8 0)	J4-15 (K14)		column 2, engine pin 5
keypad pin 7 (column 3: 3 6 9 #)	J4-16 (K16)		column 3, engine pin 6
keypad pin 8 (column 4: A B C D)	J4-17 (K15)		column 4, engine pin 7

The sketch names each key and prints the raw map. Every key mirrored (1 reads as *) = the tail is numbered the other way: swap the two groups of four. A key that never shows = its row or column lead.

IR receiver KY-022 (rerun.ps1 -Cmd m: NEC frames as address and command, 'repeat n' while a button is held; -Cmd j: raw pulse lengths) and ultrasonic HC-SR04 (5 V part, echo divided down, program to follow): one at a time, they share J4-8.

FROM	TO	AND (third end)	What it is
IR module S (signal)	J4-8 (N16)		engine pin 1: idles high, pulses low under the 38 kHz bursts
IR module + (VCC)	J4-2 (3.3 V; or J4-1)		power
IR module - (GND)	common ground		power
HC-SR04 VCC	ESP32 5 V pin (VIN)		5 V power
HC-SR04 GND	common ground		power

FROM	TO	AND (third end)	What it is
HC-SR04 Trig	J4-18 (J14)		engine pin 8 (out only): the 10 us trigger; 3.3 V is enough
HC-SR04 Echo	1 k resistor	other end to J4-8 (N16) and a 2 k resistor to GND	echo (5 V) divided to 3.3 V, engine pin 1; 58 us per cm

IR emitter board (rerun.ps1 -Cmd t, loopback to the receiver): signal to J4-19 (H14), G to GND (J4-11 or 12), middle pin not connected. Face the LED at the receiver, a few cm apart.
IR module pads are marked G R Y (ground, power, signal): Y is the S lead. The IR **emitter** (KY-005) must not share J4-8: its LED holds the line low. Pin 1 has a pull-up, so idle-high alone proves nothing about the wiring.
Manhattan remote button codes read on the kit, 4 Nov 2026 (-Cmd m, twice, same both times). NEC timing, 16-bit address and 16-bit command with no inverse, so the sketch prints BAD INVERSE for every key: not a read error. Address word 2916 on every key.

Button	Command word	Button	Command word
1	5CB7	7	5BC7
2	6CB6	8	6BC6
3	7CB5	9	7BC5
4	0CC2	0	3BC9
5	1CC1	POWER	9CB3
6	2CC0	VOL+	2AE0

8. Pico 2 as the host instead of the ESP32 (4 Nov 2026)

One host at a time: move the five SPI/IRQ leads and the ground from the ESP32 to the Pico (kit ends stay put). MicroPython v1.29.0. Files in pico/: chip_host.py, programs.py, id_test.py; copy with pico\put.ps1, run with pico un.ps1 -Script id_test.py. First test: ID 0x0022 five times.

FROM	TO	AND (third end)	What it is
J4-3 (K12)	Pico GP17 (pin 22)		CS
J4-5 (M14)	Pico GP19 (pin 25)		MOSI (SPI0 TX)
J4-7 (L14)	Pico GP16 (pin 21)		MISO (SPI0 RX)
J4-9 (M15)	Pico GP18 (pin 24)		SCK
J4-4 (K13)	Pico GP20 (pin 26)		IRQ
J4-11 (GND)	Pico GND (pin 23)		common ground, between GP18 and GP19

9. AT24C256 EEPROM module on the I2C bus (4 Nov 2026, from the Pico)

Replaces the UNO test slave on J4-10 and J4-13 (the UNO also answers at 0x50: not both). Its pull-ups go to VCC: **power it from 3.3 V only**. Jumpers A0-A2 to GND = address 0x50, WP to GND = writes allowed. Test: pico/run.ps1 -Script at24c.py. **Ground:** the board's negative rail must be wired to a kit GND hole (J4-11 or 12); for a while only the logic analyser's lead tied it in.

FROM	TO	AND (third end)	What it is
module VCC	J4-2 (3.3 V)		power; the D1 LED lights
module SCL	J4-10 (M16)		engine pin 2
module SDA	J4-13 (L15)		engine pin 3
module GND	negative rail	rail wired to J4-11 or 12	ground

10. DALI: PLANNED 4 Nov 2026, NOT YET WIRED (the gear and the shield are on order)

Chip side model-proven (programs/dali_tx.asm, dali_rx.asm, model/test_dali.py); the run is pico/dali.py from the Pico; full note docs/dali_bench.md. Interface: the **Waveshare Pico-DALI2** shield (isolated, polarity-free DALI terminal, no bus supply of its own; no Pico in it: jumper leads into its female header, pins numbered as a Pico's). Bus supply: a DALI PSU, or 12-18 V DC through 56 ohm 2 W. The gear (a DALI LED driver) across the bus. **Leads that move:** the Pico's GP0 UART lead comes out of J4-6; J4-18 is also the HC-SR04 trigger's hole, one at a time.

FROM	TO	AND (third end)	What it is
J4-18 (J14)	shield pin 9 (DALI TX)		engine pin 8, out only: high = bus shorted, idle low
shield pin 5 (DALI RX)	J4-6 (N14)		engine pin 0: bus high = 1, shorted = 0 (not pins 15-19: the kit gives them pull-downs)
J4-1 or J4-2 (3.3 V)	shield pin 36 (3V3)		logic-side power
J4-11 or J4-12 (GND)	shield pin 3 (or any GND pin)		common ground; the bus side is isolated from it
shield DALI terminal	the gear's DA terminals	the bus supply: DALI PSU, or 12-18 V through 56 ohm 2 W	the bus, any polarity
shield pin 4	not connected		the shield's bus-voltage sense

Expected printout: DAPC 254 no answer, QUERY LEVEL 254, DAPC 127, QUERY LEVEL 127, DEVICE TYPE 6, STATUS with 'lamp on', OFF, QUERY LEVEL 0: PASS. Nothing back: docs/dali_bench.md has the order of checks (pin 0 with read_pins.asm first).

11. W25Q32 SPI flash module: parts due 5 Nov 2026, NOT YET RUN (pico/w25q.py)

Engine A runs programs/spi_master.asm (mode 0, MSB first, SCK 1 MHz) to a Winbond W25Q32 module (4 MB, 3.3 V). **Why these holes:** the program takes its four pins as one group: SCK = base + 0, MOSI = base + 1, MISO = base + 2 (the input), CS = base + 3, so they must be four consecutive engine pins with the third able to read: on the kit that is pins 4-7 only (8-13 are output only, 15-19 input only, 14 is not on the header, 0-3 are taken). So the flash takes the 74HC595's holes: **the keypad's four column leads (tail 5-8) come out of J4-14 to 17 first**; the rows and their pull-ups stay. WP and HOLD are usually pulled up on the module; if bare, tie both to 3.3 V.

FROM	TO	AND (third end)	What it is
J4-14 (L16)	module CLK		SCK, engine pin 4
J4-15 (K14)	module DI (IO0)		MOSI, engine pin 5
J4-16 (K16)	module DO (IO1)		MISO, engine pin 6: the flash's data back
J4-17 (K15)	module CS		CS, engine pin 7 (idles high)
module VCC	J4-1 or J4-2 (3.3 V)		power: 3.3 V only
module GND	negative rail	rail wired to J4-11 or 12	ground
module WP, HOLD	as the module has them (pulled up on board)	or to 3.3 V if bare	both must be high

Test: pico/run.ps1 -Script w25q.py -Seconds 30. Expect JEDEC ID EF 40 16 (Winbond, 4 MB), a unique ID read twice the same, status 1 = 00, 16 bytes at address 0 as found, then the write proof: the first all-FF 16-byte region from the top of memory down is programmed with a clock-seeded pattern, BUSY polled (under 3 ms), read back equal; address 0 unchanged: PASS. Each run uses one more region; ERASE = True in the script erases the top 4 KB sector (wears the part: only when the regions run out). All 00 or FF for the ID = a lead: CS and CLK, DO, then power.

12. DS3231 clock module on the I2C bus: parts due 5 Nov 2026, NOT YET RUN (pico/ds3231.py)

The ZS-042-type module: a DS3231 at **0x68** and an AT24C32 EEPROM (4 KB, 2 address bytes, 32-byte pages) at **0x57**, pull-ups to VCC on board, so **3.3 V only**. It joins the AT24C256 (0x50) on the same two wires: no lead moves. At 3.3 V the module's LIR2032 charge path sits below a 3 V cell and does nothing, so a plain CR2032 is safe here. Two things in the drawer answer at 0x68 (this and the GY-521's MPU6050): the script reads register 0x75 (an MPU6050 answers 0x68 there) and proves the clock by its running seconds.

FROM	TO	AND (third end)	What it is
module VCC	J4-1 or J4-2 (3.3 V)		power: 3.3 V only
module GND	negative rail	rail wired to J4-11 or 12	ground
module SCL	J4-10 (M16)	the AT24C256 module's SCL stays	engine pin 2
module SDA	J4-13 (L15)	the AT24C256 module's SDA stays	engine pin 3
module 32K, SQW	not connected		unused outputs

Test: pico/run.ps1 -Script ds3231.py -Seconds 20. Expect answered 0x50 0x57 0x68; register 0x75 not 0x68; the time registers twice, two seconds apart, seconds advanced by 2 (1-3 passes); 24-hour mode; the temperature in degrees (the room); the AT24C32 page write and read back, the top of the 4 KB (0x0FF0) written without disturbing 0x0010, all put back: PASS. OSF set in status 0x0F = the clock stopped since it was set: put a tuple in SET_TIME at the top of the script.

13. MCP4725 DAC module on the I2C bus: parts due 5 Nov 2026, NOT YET RUN (pico/mcp4725.py)

A 12-bit DAC at **0x60** (0x61 with A0 high; 0x62/0x63 on an A1 part: the script probes 0x60-0x67), on the same two wires, the master at 400 kHz. Output = VDD x code / 4096: with the 3.3 V supply 0, 1.65 and 3.30 V for codes 0, 2048 and 4095. Meter OUT against the rail; the scope on OUT for the ramps. Fast-mode writes leave the DAC's EEPROM alone.

FROM	TO	AND (third end)	What it is
module VDD (VCC)	J4-1 or J4-2 (3.3 V)		power: 3.3 V only; the output's full scale
module GND	negative rail	rail wired to J4-11 or 12	ground
module SCL	J4-10 (M16)	the other modules' SCL stay	engine pin 2
module SDA	J4-13 (L15)	the other modules' SDA stay	engine pin 3
module OUT	meter red lead / scope CH1	black lead / probe ground to the rail	the DAC output
module A0	as the module has it		address low bit: open or low = 0x60

Test: pico/run.ps1 -Script mcp4725.py -Seconds 40. Expect the address found; 0 V, 1.65 V, 3.30 V held 3 s each for the meter; a 64-step sawtooth 10 s and a triangle 10 s, every byte acked: PASS. The ramp rate is host-limited (a round trip per byte): the script measures the ramps it made and prints the Hz the scope must show (the wire alone would allow 347 Hz at 400 kHz). Scope: CH1 on OUT, 1 V/div, trigger on the sawtooth's falling flyback.

14. Mojo v3 with the IO Shield stacked, Pico as host (5 Nov 2026; regression PASS)

Three engines at 50 MHz (ID **0x0023**). Leads go into the IO Shield's stacking headers; **the number printed beside a hole is that hole's FPGA pin**. Right header: inner hole (towards the display) labelled on the inside (50, 40, 34 ...), outer hole on the board edge (51, 41, 35 ...); row 1 is inner RAW / outer GND, row 2 inner +V / outer GND. Left header: inner GND, GND, VSYS, 58, 67, 75 ...; outer RAW, +V, DONE, 57, 66, 74 ... **All 24 shield DIP switches OFF.** Load: python fpga/mojo/load.py COM14 fpga/mojo/build/chip.bin (FPGA RAM only). **After any power blip or reset press the Mojo runs its flash design** (LEDs fade, ID garbage): reload.

FROM	TO	AND (third end)	What it is
Pico GND (pin 23)	right header row 1, OUTER hole, edge label GND		common ground (the inner hole of that row is RAW)
Pico GP17 (pin 22)	right header, inner hole 50		CS
Pico GP18 (pin 24)	right header, outer hole 51		SCK
Pico GP19 (pin 25)	right header, inner hole 40		MOSI
Pico GP16 (pin 21)	right header, outer hole 134 (4th row from the far end)		MISO (also Mojo LED0)
Pico GP20 (pin 26)	right header, inner hole 133 (same row)		IRQ (also Mojo LED1)
Pico GP1 (pin 2)	right header, outer hole 41		engine pin 0: UART out into the Pico's UART0 RX
Pico GP2 (pin 4)	left header, outer hole 57 (4th row)		engine pin 15: test pulses from the Pico

Engine pins (hole label = FPGA pin): 0-7 = 41, 34, 35, 32, 33, 29, 30, 27 (right header; I2C SCL/SDA 35/32; SPI SCK/MOSI/MISO/CS 33/29/30/27); 8-13 = 132, 131, 127, 126, 124, 123 (right header, the Mojo's LED2-LED7 light with them); 15-19 = 57, 58, 66, 67, 74 (left header, DIP switches OFF).

Tests: pico/run.ps1 -Script id_test.py (0x0023 five times), then -Script regression.py -Seconds 40 (CHIP_HZ = 50_000_000): PASS, 0 problems. Nothing back: -Script lead_probe.py (MISO and IRQ driven and steady; toggling = the flash design is running), then the pin finder (TOP=pin_finder build: touch a hole with a ground lead through 220 R - 1 k, the display shows P and the pin number).